

[Sign In](#)[Free Startup Course](#)

Scaling Flow in a Startup



ASH MAURYA
BOOTSTRAPPING
FOCUS

Last year, I outlined ten work hacks I use for managing conflicting pulls on my time in a post titled: [“Achieving Flow in a Lean Startup.”](#) This is a follow-up to that post, where I’ll describe how I’ve been scaling these tactics since then.

The basic problem is that as a founder, you need to be involved in everything—customer development, product development, administration, etc. And that’s just for your startup—You also have a life outside your startup (or should).

Organizing my daily, weekly, and monthly routines to flow was a way to balance my “inside the building” versus “outside the

Subscribe to the Newsletter

Join thousands of founders for battle-tested recipes, strategies, and how-tos for achieving product/market fit systematically.

Send
Me
Weekly
Startup
Tips →

SHARE THIS ARTICLE ON:



building” activities while maintaining sanity. But as my “outside the building” activities have continued to grow, I’ve found it increasingly necessary to find ways to scale beyond myself.

The Nature of Work

All work can be accomplished as a function of time, effort, and money. Of the three, **time is your most valuable resource** because it only moves in one direction. Some work is time-sensitive (e.g. paying bills and filing taxes), while other tasks need to be optimized for time (e.g. learning about customers).

The end result of all work should be moving you towards value generation. While value needs to ultimately be measured in terms of money (if you are to build a sustainable business), you initially rely on other key metrics (earlier in the user adoption funnel).

Achieving flow is about identifying value-creating activities and organizing them to reduce on waiting and context switching.

Scaling flow is about optimizing these value-creating activities and getting more done through allocation, delegation, automation, and leadership.

Stage 1: Maximizing Yourself —Allocation

I recently read Ramit Sethi’s book: [“I Will Teach You to Be Rich”](#) where he describes some interesting money management tactics.



The basic premise is that budgeting doesn't work because people aren't disciplined enough to stick to them—There are way too many things to keep track of.

Ramit instead advocates allocating your money into four different buckets (fixed expenses, long-term savings, short-term savings, and guilt-free spending) and then **automating their allocation** (using automatic bank transfers). For most people, saving is an afterthought, and because it's at the bottom of their "money funnel," they never get to it. By inverting the funnel, you save first and what you're left with is guilt-free spending money that doesn't require a budget. It's a great system to get you 80% of the way there.

The same approach can also be applied to managing your time and effort, so you pre-allocate your value-creating activities first.

Here's how I break up my buckets—building, learning, administrative, and others.

Building (35%): Anything I create under "maker rules" falls in this category. This includes writing code, blogging, designing a screen, copywriting, etc. The outcome of these activities is tangible products.

Learning (35%): Any conversation with or about customers is a learning activity. These may be direct conversations with customers, such as interviews, support calls, etc., or indirect conversations about customers, such

as weekly status meetings, analyzing metrics, etc. The outcome of these activities is actionable learning (falsifiable hypotheses) that drives our next set of experiments (initiated with additional building activities).

Administrative (10%): This is for routine tasks, that, while otherwise boring, are critical to the operation of the startup, such as paying bills, invoicing, backing up server databases, etc.

Other (20%): This is my “guilt-free” bucket for everything else e.g., other forms of learning, meetings, events, etc.

I allocate these activities as blocks of time into my daily, weekly, and monthly routines and make sure they are organized to flow. The [“Achieving Flow” post](#) describes how.

Additional Notes:

- I also have several non-negotiable “life” routines, such as dropping my kids to school, yoga, cooking, etc., that, while not “work,” enforce constraints on my time and shape my day.
- I need to make time to think about my schedule. Some of my best ideas have come from an idle mind.

“The only place that work and motion are the same thing is the zoo where people pay to see the animals move

around.”

– Taiichi Ohno

Stage 2: Scaling Yourself—Automation, Delegation, and Outsourcing

Allocating your schedule around value generating activities gets you 80% there. Now it's time to revisit these activities and optimize further.

I start by further classifying the work into the following categories: unique, core, and non-core.

Unique: These are tasks I wouldn't delegate to anyone else, e.g. blogging on my personal site, teaching workshops, writing the [Running Lean Mastery blog](#), interviewing customers, etc. This list reflects the unique value proposition you bring to your startup. The list will probably change over time as you begin to develop systems for doing these things.

Core: These are tasks that are core to the unique value proposition of your startup but aren't necessarily unique to you, i.e., they can be delegated to others, e.g., designing screens, writing code, etc.

Non-core: This group is for non-core tasks that may be outsourced or automated e.g. scheduling meetings, accounting, testing, etc.

The “unique” tasks stay on my plate, and I look for ways to optimize the others in the following order:

1. Automate routine tasks

Spending a little effort up front to automate routine tasks goes a long way. For example, I spent a whole day every other week on software releases which amounted to 10% of my time every month. Now I rely heavily on our continuous deployment system to build, test, deploy, and monitor our products. The whole system is automated and unless there is an issue, pushing code to production is a non-event. The 5-Whys approach is effective in uncovering these types of recurring tasks.

2. Outsource non-core tasks

Anything that can be affordably outsourced should.

I find that most people grossly undervalue their time, especially when it comes to the form of sweat equity. Even if you aren't yet paying yourself, I find it helpful to put a dollar value to your time that is at least your market rate. Then use that to justify building in-house versus outsourcing.

I migrated my last company's infrastructure to Amazon EC2 which was a huge leap over hosting our servers, but it still took quite a bit of work to keep running. With my latest startup, I immediately got on Heroku. While comparatively more expensive than running

directly on EC2, Heroku seamlessly handles scaling, deployments, and configuration which frees up more of my time to work on core tasks. Your first objective should be getting to a plan that works as quickly as possible. You can always revisit optimizing your cost structure later.

The same applies to non-core human tasks. The two biggest sources of waste that affect productivity are waiting and context switching. One of my pet peeves is scheduling meetings. I've tried automated systems (like tungle) which work fine until they don't. It's not that these are big tasks but they are interrupt-driven and big enough to cause a context switch. I've been experimenting with a virtual assistant and have been successful with moving a lot these administrative type of tasks, like scheduling, reservations, expense reporting, etc., off my plate.

I've also effectively used oDesk for small non-core coding tasks, such as building a dashboard for my book sales. These are tasks that while I can do them myself, are not value-generating enough to justify my time.

3. Delegate core tasks

I described our Kanban board for tracking the feature lifecycle [in a previous post](#). It's a great system for both visualizing and delegating work to others. As I mentioned earlier, I do not delegate "customer learning" activities at the moment (which are the green areas on the board), only building activities.

Since building and learning intertwine heavily, I prefer to only delegate these tasks to other people on “the team” as opposed to outsourcing to someone on oDesk. I don’t have a dedicated full-time team on my current startup but instead rely on a team of “specialists” made up of other Lean Startup practitioners.

Here’s what I’ve learned so far:

Entrepreneurs make great specialists

I’ve outsourced lots of small and big projects, and while the quality of the work has been mostly acceptable, there is constant management overhead to get these tasks done. Last year, I put out a call for help on my blog to work on what was then a “toy” side-project: Lean Canvas. A number of entrepreneurs answered my call which is how I got the Lean Canvas MVP built.

The pace, quality, and ease of working with other entrepreneurs were phenomenal. We instantly understood each other. Meetings were short and productive. Issues got identified and resolved. Things just got done.

It helped that all of them were already interested in the “problem domain” and wanted to be early users/customers, too—This wasn’t “just a job” to them.

I was so pleased with this mode of working that I extended the call for help and put



together a virtual practitioners team that spans four continents.

Everyone builds

A while back, there was an “everyone codes” meme going around the web. While I subscribe to the general idea, I don’t think “build” activities are just limited to coding. I count mockups, copy, blog posts, etc. part of building. I believe everyone on the team needs to build something tangible that can then be measured. Simply participating in learning activities is not enough. I’ve got advisors for that.

Working in small batches leads to less waste

Apart from myself, everyone else is part-time. I’ve actually found this to result in less waste. In my last company, I negotiated a discounted rate from my outsourcing company in India in exchange for a regular number of hours each week. Because I had to use these hours, I went into “build overload,” which resulted in lots of unused features and a bloated product.

In the new Kanban system, we constrain the number of features we work on at any given time, and I run all the customer learning checkpoints, which further constrains the pace of the work and ensures that we can only go as fast as we learn.

Paying in cash is the cleanest option

I’ve made the mistake of trying to co-found a company too soon with people I thought would work out -only to be proven wrong.



Startups are faced with enough uncertainty that I find it best to treat everything as an experiment, including the team you build. Rather than tying yourself up in complicated agreements or hard-to-value equity distributions, I find it much simpler to pay-as-you-go—at least at first.

This also places real constraints on your budget, which forces more thorough vetting to ensure that a) you have the right team in place and b) that you are asking them only to build things worth building.

Stage 3: Scaling Beyond Yourself—Leadership

I am currently mostly operating in Stage 2 but have started thinking about practices for building and scaling a learning organization beyond the core founding team.

Going from everyone builds to everyone experiments

Stage 2 is about optimizing the efficiency of the founding team by building an extended “specialists” team, but the founding team still constrains the pace of learning. To scale beyond the founding team, the extended team culture needs to shift from being specialists to experimenters.

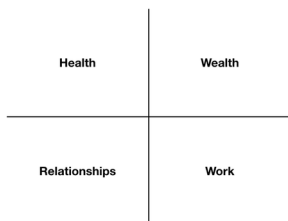
In this model, the next key metric goals are clearly declared upfront and anyone on the team is free to formulate hypotheses and initiate experiments to test them.



Eric Ries describes a new concept of “Innovation Accounting” in his upcoming book: The Lean Startup which I find fascinating. While I don’t have concrete practices yet to share, this is an area I’ll be exploring more.

“The Toyota style is not to create results by working hard. It is a system that says there is no limit to people’s creativity. People don’t go to Toyota to ‘work’ they go there to ‘think’”
– Taiichi Ohno

Read Next:



The True Value of Your Time

Time is our scarcest resource. Other resources like money and people can fluctuate up and down, but time only moves in one direction.

Ready to Start Your Lean Journey with CI Foundations?

Join 25,000+ founders who've shortcut their path to customers